



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Ingeniería en Computación

Materia: Estructura de Datos y Algoritmos II

Proyecto: Visualizador Interactivo de Algoritmos

Marco Teórico de Algoritmos y Estructuras de Datos

Volumen II — Fundamentos Teóricos y Análisis de Complejidad

Equipo de Desarrollo - Backend:

- Francisco José Coutiño Morales (*Lead Backend Developer*)
- Ernesto Flamenco Villaseñor (*Backend Developer*)
- Tlalli Ortega González (*Backend Developer*)
- Sergio Alonso Salcedo Sainz (*Backend Developer*)
- Orlando Ocampo Lagunas (*Backend Developer*)
- Aarón Salazar Castillo (*Backend Developer*)
- Erick Cruz Solís (*Backend Developer*)
- Augusto Gandarilla Ibarra (*Backend Developer*)
- Neri Israel Cruz Cárdenas (*Backend Developer*)

Tipo: Material Didáctico de Apoyo Académico

Periodo: 2026 – 2

Índice

1. Algoritmos de Ordenamiento	2
1.1. Fundamentos del Ordenamiento	2
1.2. Bubble Sort	2
1.3. Heap Sort y Estructura Heap	2
1.4. Quick Sort y Partición	3
1.5. Counting Sort — No Comparativo	3
1.6. Radix Sort — Ordenamiento por Dígitos	3
1.7. Merge Sort — Divide y Vencerás	3
1.8. Comparativa General	4
2. Algoritmos de Búsqueda	4
2.1. Fundamentos de la Búsqueda	4
2.2. Búsqueda Lineal	4
2.3. Búsqueda Binaria	4
2.4. Funciones Hash y Tablas de Dispersión	5
2.5. Resolución de Colisiones	5
3. Algoritmos de Grafos	5
3.1. Teoría de Grafos — Definiciones Fundamentales	5
3.2. Representaciones de Grafos	5
3.3. BFS — Búsqueda por Amplitud	6
3.4. DFS — Búsqueda en Profundidad	6
4. Árboles	6
4.1. Conceptos Fundamentales	6
4.2. Árboles Binarios de Búsqueda (BST)	6
4.3. Recorridos de Árboles	7
4.4. Notaciones Aritméticas	7
4.5. Árboles B y B+	7
5. Archivos y Organización de Datos	7
5.1. Organización Lógica y Física	7
5.2. Métodos de Acceso	8
6. Introducción a los Algoritmos Paralelos	8
6.1. Modelos de Paralelismo y Granularidad	8
6.2. Modelo PRAM	8
6.3. Análisis de Desempeño — Trabajo y Profundidad	8

1. Algoritmos de Ordenamiento

1.1. Fundamentos del Ordenamiento

Definición (Algoritmo de Ordenamiento): Dado un arreglo $A[1..n]$, un algoritmo de ordenamiento produce una permutación $A'[1..n]$ tal que $A'[1] \leq A'[2] \leq \dots \leq A'[n]$.

Los algoritmos de ordenamiento se clasifican según:

- **Estabilidad:** preserva el orden relativo de elementos con claves iguales. Merge Sort y Counting Sort son estables; Quick Sort y Heap Sort generalmente no.
- **In-place:** usa $O(1)$ memoria auxiliar. Bubble, Heap y Quick Sort son in-place; Merge Sort requiere $O(n)$.
- **Cota inferior comparativa:** cualquier algoritmo basado en comparaciones necesita $\Omega(n \log n)$ comparaciones en el peor caso (demostrable por el árbol de decisión).

1.2. Bubble Sort

Definición (Bubble Sort): Compara pares adyacentes $(A[j], A[j + 1])$ e intercambia si $A[j] > A[j + 1]$. En cada pasada i , el i -ésimo elemento mayor queda en su posición definitiva.

Propiedad (Invariante de ciclo): Después de la k -ésima pasada, los k elementos mayores están en sus posiciones definitivas al final del arreglo. La lista queda ordenada en a lo más $n - 1$ pasadas.

Complejidad: $T_{mejor} = O(n)$, $T_{promedio} = T_{peor} = O(n^2)$, $S = O(1)$. Su única ventaja práctica es detectar arreglos ya ordenados con una bandera de intercambio.

1.3. Heap Sort y Estructura Heap

Definición (Max-Heap): Un Max-Heap es un árbol binario completo donde para todo nodo i : $A[\text{padre}(i)] \geq A[i]$. En una representación de arreglo, el hijo izquierdo de i está en $2i + 1$ y el derecho en $2i + 2$.

Propiedad (Construcción de heap): El algoritmo de Floyd construye un Max-Heap en $O(n)$ tiempo, más eficiente que n inserciones individuales que costarían $O(n \log n)$.

Fase de extracción: $n - 1$ operaciones Heapify de $O(\log n)$ cada una, resultando en $O(n \log n)$ total. Garantiza este tiempo en *todos* los casos, a diferencia de Quick Sort.

1.4. Quick Sort y Partición

Definición (Partición de Lomuto): Dado un arreglo $A[bajo..alto]$ con pivote $p = A[alto]$, la partición reorganiza el arreglo tal que todos los elementos $\leq p$ quedan a la izquierda del pivote y todos los $> p$ a la derecha.

Recurrencia promedio:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) \implies T(n) = O(n \log n)$$

cuando el pivote divide el arreglo en partes iguales. En el peor caso (arreglo ordenado con pivote = último elemento): $T(n) = T(n-1) + O(n) = O(n^2)$.

Nota (Estrategias de elección de pivote): *Último elemento:* simple pero $O(n^2)$ en arreglos ordenados. *Mediana de tres:* toma la mediana del primero, medio y último. *Pivote aleatorio:* esperanza $O(n \log n)$ con alta probabilidad.

1.5. Counting Sort — No Comparativo

Definición (Counting Sort): Para enteros en $[0, k]$, cuenta ocurrencias, acumula prefijos y construye el arreglo de salida en tiempo $O(n+k)$ y espacio $O(k)$.

Rompe la cota $\Omega(n \log n)$ al explotar que los elementos son enteros en rango acotado, no realizando comparaciones entre ellos. Óptimo cuando $k = O(n)$. Estable si se procesa el arreglo de entrada en orden inverso.

1.6. Radix Sort — Ordenamiento por Dígitos

Ordena números dígito a dígito (LSD a MSD) usando un algoritmo estable (Counting Sort) como subrutina. Para d dígitos en base b : $O(d(n+b))$.

Nota (Cuando usar Radix Sort): Cuando d es constante y $b = O(n)$, Radix Sort logra $O(n)$. Muy usado para cadenas de longitud fija, números IP, y ordenamiento de registros en bases de datos.

1.7. Merge Sort — Divide y Vencerás

Definición (Merge Sort): Divide $A[1..n]$ en $A[1..n/2]$ y $A[n/2+1..n]$, ordena cada mitad recursivamente y fusiona los resultados. La fusión de dos secuencias ordenadas de tamaño $n/2$ requiere exactamente $O(n)$ comparaciones.

Recurrencia:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) \implies T(n) = O(n \log n)$$

(caso 2 del Teorema Maestro). Estable y predecible: garantiza $O(n \log n)$ en todos los casos. Estándar para *ordenamiento externo*.

Algoritmo	Mejor	Promedio	Peor	Espacio	Estable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sí
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	Sí
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n + k)$	Sí
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sí

Cuadro 1: Comparativa de complejidad — Algoritmos de ordenamiento implementados

1.8. Comparativa General

2. Algoritmos de Búsqueda

2.1. Fundamentos de la Búsqueda

Definición (Problema de búsqueda): Dado un conjunto S y un elemento objetivo x , determinar si $x \in S$ y, en caso afirmativo, su ubicación en la estructura de datos.

Clasificación según estado de los datos:

- **Datos no ordenados:** búsqueda lineal, $O(n)$ inevitable (cota inferior por argumento del adversario).
- **Datos ordenados:** búsqueda binaria $O(\log n)$, interpolación $O(\log \log n)$ promedio.
- **Transformación de clave:** hashing, $O(1)$ amortizado.

2.2. Búsqueda Lineal

Examina cada elemento secuencialmente. Complejidad $O(1)$ mejor caso, $O(n)$ peor y promedio. Óptima cuando los datos no están ordenados (cota inferior $\Omega(n)$ por el argumento del adversario).

2.3. Búsqueda Binaria

Definición (Búsqueda Binaria): Sobre $A[1..n]$ ordenado, compara $A[\lfloor (iz + der)/2 \rfloor]$ con x . Descarta la mitad donde x no puede estar en cada paso.

Recurrencia: $T(n) = T(n/2) + O(1) \implies T(n) = O(\log n)$. Con $n = 10^9$: máximo 30 comparaciones.

Propiedad (Invariante): Si $x \in A$, entonces $x \in A[izquierda..derecha]$ al inicio de cada iteración.

2.4. Funciones Hash y Tablas de Dispersión

Definición (Función Hash): Una función $h : U \rightarrow \{0, 1, \dots, m-1\}$ mapea un universo de claves U a índices de una tabla de tamaño m . Una colisión ocurre cuando $h(k_1) = h(k_2)$ para $k_1 \neq k_2$.

Factor de carga: $\alpha = n/m$. Para $\alpha < 0,75$, las operaciones tienen $O(1)$ esperado. Al superar este umbral, se realiza una rehashificación que duplica la tabla, costando $O(n)$ amortizado a $O(1)$ por operación.

2.5. Resolución de Colisiones

Encadenamiento: listas enlazadas en cada posición. Tiempo de búsqueda $O(1 + \alpha)$ promedio. Tolerancia $\alpha > 1$.

Sondeo lineal: $h(k, i) = (h(k) + i) \bmod m$. Sufre de *agrupamiento primario* que degrada el rendimiento cuando α es alto.

Doble hashing: $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$. Elimina el agrupamiento primario. $h_2(k)$ debe ser coprimo con m .

3. Algoritmos de Grafos

3.1. Teoría de Grafos — Definiciones Fundamentales

Definición (Grafo): Un grafo $G = (V, E)$ consiste en un conjunto finito de vértices V y un conjunto de aristas $E \subseteq V \times V$. Si las aristas tienen dirección, G es un *dígrafo*; en caso contrario es *no dirigido*.

Terminología esencial:

- **Grado** $\deg(v)$: número de aristas incidentes en v . En dígrafos: grado de entrada $\deg^-(v)$ y salida $\deg^+(v)$.
- **Camino**: secuencia de vértices $v_0, v_1, \dots, v_k$ donde $(v_{i-1}, v_i) \in E$. Si $v_0 = v_k$, es un *ciclo*.
- **Conexidad**: G es conexo si existe camino entre todo par de vértices.
- **Árbol**: grafo conexo acíclico con $|V| - 1$ aristas.

3.2. Representaciones de Grafos

Representación	Espacio	Verificar arista	Vecinos de v	Óptima para
Matriz de adyacencia	$O(V^2)$	$O(1)$	$O(V)$	Grafos densos
Lista de adyacencia	$O(V + E)$	$O(\deg(v))$	$O(\deg(v))$	Grafos dispersos

Cuadro 2: Comparativa de representaciones de grafos

3.3. BFS — Búsqueda por Amplitud

Definición (BFS): Dado un grafo G y un vértice fuente s , BFS visita todos los vértices alcanzables desde s en orden no decreciente de distancia (en aristas).

Propiedad (Árbol BFS): Al finalizar BFS, el árbol de exploración contiene caminos más cortos (en número de aristas) desde s hacia todo vértice alcanzable.

Complejidad: $O(V + E)$ tiempo, $O(V)$ espacio.

Aplicaciones: camino más corto no ponderado, detección de bipartición, nivel de separación en redes sociales.

3.4. DFS — Búsqueda en Profundidad

Definición (DFS): Explora el grafo profundizando en cada rama antes de retroceder. Asigna tiempos de descubrimiento $d[v]$ y finalización $f[v]$ a cada vértice.

Clasificación de aristas en DFS:

- **Árbol:** seguidas durante la exploración.
- **Retroceso:** hacia un ancestro; indican ciclos en dígrafos.
- **Avance/cruzadas:** solo en dígrafos.

Aplicaciones: detección de ciclos, ordenamiento topológico, componentes fuertemente conexas (Tarjan, Kosaraju).

4. Árboles

4.1. Conceptos Fundamentales

Definición (Árbol con raíz): Un árbol con raíz es un grafo acíclico conexo con un vértice distinguido llamado raíz. Define una relación jerárquica padre-hijo entre vértices.

Terminología: *altura* = longitud del camino más largo raíz-hoja; *profundidad* de v = distancia desde la raíz; *hoja* = nodo sin hijos; *grado* de un nodo = número de hijos.

4.2. Árboles Binarios de Búsqueda (BST)

Definición (BST): Para todo nodo n con clave k :

$$\forall x \in \text{subárbol izquierdo} : \text{clave}(x) < k \quad \text{y} \quad \forall y \in \text{subárbol derecho} : \text{clave}(y) > k$$

Complejidad de operaciones: $O(h)$ donde h es la altura. En el peor caso (secuencia ordenada): $h = n$, degenerando a lista enlazada. Árboles balanceados (AVL, Red-Black) garantizan $h = O(\log n)$.

Recorrido	Orden de visita	Aplicación principal
Inorden	Izq – Raíz – Der	Lista ordenada de un BST
Preorden	Raíz – Izq – Der	Serialización del árbol
Postorden	Izq – Der – Raíz	Eliminación, evaluación sufija

Cuadro 3: Recorridos de árboles binarios

4.3. Recorridos de Árboles

4.4. Notaciones Aritméticas

Definición (Notaciones):

- **Infija:** $A + B$ — operador entre operandos, requiere paréntesis.
- **Prefija (polaca):** $+AB$ — operador antes de operandos.
- **Sufija (RPN):** $AB+$ — operador después de operandos.

La conversión infija $\rightarrow$ sufija se realiza con el algoritmo de Shunting Yard de Dijkstra en $O(n)$. La evaluación de expresiones sufijas usa una pila en $O(n)$ sin necesidad de manejar paréntesis ni precedencia explícita.

4.5. Árboles B y B+

Definición (Árbol B de orden m): Árbol de búsqueda balanceado donde cada nodo tiene entre $\lceil m/2 \rceil$ y m hijos. Todos los nodos hoja están al mismo nivel. Diseñado para minimizar accesos a disco.

Propiedad (Árbol B+): En un árbol B+, todos los datos se almacenan en las hojas, que se enlazan formando una lista doblemente enlazada. Permite recorridos secuenciales eficientes. Estándar en sistemas de bases de datos (MySQL InnoDB, PostgreSQL).

5. Archivos y Organización de Datos

5.1. Organización Lógica y Física

Definición (Organización de archivos): La *organización lógica* describe la vista abstracta de los datos (registros, campos). La *organización física* describe la disposición real en el dispositivo de almacenamiento (sectores, bloques).

Tipos de organización:

- **Secuencial:** registros almacenados consecutivamente. Óptimo para procesamiento batch, $O(n)$ para búsqueda aleatoria.
- **Indexado:** índice separado (típicamente Árbol B+) que mapea claves a posiciones físicas. Búsqueda en $O(\log n)$ accesos a disco.
- **Acceso directo (hash):** la posición física se calcula mediante una función hash. Búsqueda $O(1)$ promedio.

5.2. Métodos de Acceso

El **acceso lógico** abstrae los detalles físicos; el programador trabaja con registros. El **acceso físico** manipula bloques directamente, usado en sistemas de bases de datos que implementan su propio buffer pool.

Nota (Jerarquía de memoria y latencia): Registro CPU: ~ 1 ns; Cache L1: ~ 1 ns; RAM: ~ 100 ns; SSD: ~ 0.1 ms; Disco magnético: ~ 10 ms. Minimizar accesos a disco es crítico: un algoritmo con $O(\log n)$ accesos a disco supera en práctica a uno con $O(n)$ accesos a RAM.

6. Introducción a los Algoritmos Paralelos

6.1. Modelos de Paralelismo y Granularidad

Definición (Algoritmo paralelo): Un algoritmo paralelo resuelve un problema usando P procesadores que ejecutan instrucciones simultáneamente, reduciendo el tiempo total (span) a cambio de mayor trabajo total o complejidad de implementación.

Ley de Amdahl: Si una fracción p del programa puede paralelizarse:

$$S(P) = \frac{1}{(1-p) + \frac{p}{P}} \xrightarrow{P \rightarrow \infty} \frac{1}{1-p}$$

El componente serial limita fundamentalmente la aceleración máxima.

6.2. Modelo PRAM

Definición (PRAM): *Parallel Random Access Machine*: P procesadores con memoria compartida de acceso en $O(1)$. Ejecutan instrucciones de forma síncrona.

Variantes según acceso a memoria:

- **EREW:** lecturas y escrituras exclusivas. El más restrictivo.
- **CREW:** lecturas concurrentes, escrituras exclusivas. El más común.
- **CRCW:** lecturas y escrituras concurrentes; las escrituras simultáneas se resuelven por prioridad, OR, o arbitrario.

6.3. Análisis de Desempeño — Trabajo y Profundidad

Definición (Trabajo y Profundidad):

- $W(n)$: **trabajo** — número total de operaciones de todos los procesadores.
- $D(n)$: **profundidad/span** — longitud del camino crítico (operaciones que deben ejecutarse secuencialmente).

Propiedad (Ley de Brent): Con P procesadores, el tiempo de ejecución satisface:

$$T_P \leq D(n) + \frac{W(n)}{P}$$

El **paralelismo máximo** es $W(n)/D(n)$, que indica cuántos procesadores pueden mantenerse útiles simultáneamente.

Ejemplo — Suma paralela por reducción en árbol:

$$W(n) = O(n) \quad D(n) = O(\log n) \quad \text{Paralelismo} = \frac{O(n)}{O(\log n)} = O\left(\frac{n}{\log n}\right)$$

Nota (Eficiencia paralela): $E = \frac{W(n)}{P \cdot T_P}$. Un algoritmo eficiente maximiza $E \rightarrow 1$, minimizando tiempo de sincronización, comunicación y procesadores ociosos.